Roteiro da Entrega 2

Projeto Reforça | gestão de mentoria escolar voluntária

Plano de trabalho para Mateus, Felipe e André, baseado no escopo, nas responsabilidades e nas cinco sprints da Entrega 1. A aplicação será Java com JDBC e MySQL, organizada em MVC por camadas, com interface de linha de comando.

Objetivo e prioridade

Entregar uma aplicação executável que organize alunos, mentores, disciplinas, planos e sessões de mentoria. A demonstração deve provar o CRUD completo da entidade principal SessaoMentoria, a regra que bloqueia conflito de horário, a gravação real no MySQL e os pilares de POO descritos no projeto.

Critério da Entrega 2	Evidência no Reforça
Aderência ao plano	Implementar os requisitos 01–09 e conservar o fluxo previsto nas cinco sprints.
POO	Usuario abstrata com Aluno, Mentor e Coordenador; SessaoMentoria abstrata com SessaoPresencial e SessaoOnline; métodos sobrescritos com @Override.
CRUD e banco	Criar, consultar, editar e cancelar/remover logicamente SessaoMentoria via JDBC conectado ao MySQL. SQL de criação em /database.
Execução e equipe	Aplicação de terminal executa conforme README em Windows/Linux; commits individuais dos três integrantes.

Divisão por pessoa e sprints

Responsável	Entrega que lidera	Tarefas concretas
Mateus	Sprint 1 e apoio na Sprint 4	Criar Usuario e subclasses Aluno, Mentor e Coordenador; cadastro/consulta e CRUD de alunos e mentores; dados específicos e validações. Na Sprint 4, implementar PlanoDeEstudo e sua composição 1:N com sessões.
Felipe	Sprint 2 e consultas da Sprint 5	Implementar CRUD de Disciplina e mentor_disciplina; agendamento e alteração de sessões; view/menu de terminal para esses fluxos. Na Sprint 5, filtros de consulta por período, mentor e disciplina e total de horas realizadas.
André	Sprint 3 e integração técnica da Sprint 5	Implementar regra de conflito no Service e acesso JDBC/DAO associado; testar intervalos sobrepostos. Integrar as subclasses SessaoPresencial/SessaoOnline com @Override, conexão MySQL, script SQL, tratamento de cancelamento com motivo e instruções de execução no README.

Compartilhado pelos três: definir contratos Model/DAO/Service/View antes da integração, revisar o código de outro integrante, fazer commits identificáveis no repositório e participar da demonstração. O responsável lidera a entrega; não trabalha isolado.

Escopo funcional para demonstrar

Req.	Comportamento concluído
01–03	Cadastrar e consultar alunos, mentores e disciplinas com os dados definidos na Entrega 1; associar mentores às disciplinas e guardar suas disponibilidades.
04	Agendar sessão com aluno, mentor, disciplina, data, início, término e modalidade presencial/online.
05	Recusar sobreposição de sessão do mesmo mentor no mesmo dia e mostrar mensagem de indisponibilidade.
06–07	Criar plano por aluno com objetivo e período; associar sessões; registrar presença, evolução de 1 a 5 e comentário do mentor.
08	Alterar sessão; cancelar com motivo obrigatório. Preservar o registro e o motivo para consulta da coordenação.
09	Filtrar sessões por período, mentor e disciplina; totalizar horas realizadas, contando apenas sessões realizadas.

Ordem técnica de integração

1. Começar pelo esquema do banco e pelos contratos das classes. Confirmar nomes de campos e relações: usuários por tipo, disciplinas e associação mentor_disciplina, planos_estudo, sessoes e registros_sessao.
2. Completar os modelos Java e o CRUD base de SessaoMentoria. Para evidenciar Delete sem perder histórico, usar cancelamento lógico com motivo obrigatório e status CANCELADA.

3. Implementar a checagem de conflito no Service antes de inserir ou alterar uma sessão. Há conflito se a sessão existente começa antes do fim solicitado e termina depois do início solicitado, para o mesmo mentor e data; ignorar sessões canceladas e a própria sessão durante edição.
4. Persistir cada operação via DAO/JDBC. Não usar listas em memória como armazenamento final. Conferir chaves estrangeiras, estados e transações no MySQL.
5. Integrar o menu de terminal, relatórios, README e script SQL; compilar e percorrer a demonstração do início ao fim em um banco limpo.

Plano de recuperação dentro da Sprint 5

A Sprint 5 vai de 09/11 a 22/11 e está em andamento em 11/11. O documento não informa quais etapas anteriores já foram concluídas; comecem marcando cada item como pronto, parcial ou pendente após abrir o repositório.

Período sugerido	Foco e saída verificável
11/11	Mateus, Felipe e André conferem o repositório, compilação, banco e progresso das Sprints 1–4. Criam lista curta de lacunas e atualizam o README/cronograma.
12–15/11	Fechar dados e relações; concluir CRUD de Sessao, agendamento e validação de conflitos; persistir no MySQL.
16–18/11	Completar plano e resultado da sessão, cancelamento com motivo e consultas/filtros/total de horas.
19–20/11	Integração final no terminal; corrigir divergências entre Java e SQL; validar o roteiro de execução em instalação limpa.
21–22/11	Congelar escopo, atualizar README e publicar arquivos faltantes; cada integrante confirma commits; ensaiar a demonstração final.

Checklist do GitHub e entrega

- Criar e manter /src, /docs e /database. Colocar código Java em /src, diagramas de classes e DER em /docs, script de criação do MySQL em /database.
- Atualizar README para refletir o projeto real: requisitos 01–09, cronograma, responsáveis, Java/JDBC/MySQL, dependências, configuração do banco e passo a passo para executar.
- Revisar se o DER corresponde às classes e ao SQL; não representar conflito de faixa horária como uma simples chave única de mentor/data/início. A validação precisa comparar intervalos.
- Confirmar acesso público ao repositório e registrar contribuições de Mateus, Felipe e André com commits próprios.